

CSC343H5 Lecture 6

Relational Algebra (cont.d) + Indexes

Naaz Sibia (naaz.sibia@utoronto.ca)

Admin

- PCRS

Me at the start of the term:

Course Philosophy: Students First

I believe the best learning happens when students feel supported, not stressed. A few principles that shape how this course is run:



Admin

- PCRS

Me at the start of the term: PCRS, Quercus, MarkUs, Chatbot

Course Philosophy: Students First



When students feel supported, not stressed. A few
s run:



**I'm afraid my brain has left
for the day.**

Admin

- PCRS

Links are now fixed, the deadline for last week's PCRS (Week 5 Practice) is now indefinite.



**I'm afraid my brain has left
for the day.**

Admin

- Break Next Week (2 weeks)
 - Go touch grass!

Me, for two weeks:



Admin

- Tutorial this week has a test problem
 - There is no midterm this week!



Recap Quiz

Relational Algebra Continued

AGGREGATION IN RELATIONAL ALGEBRA ($\mathcal{G}$)

Syntax

grouping attrs $\mathcal{G}$ *agg functions* (**R**)

Omit grouping attrs $\rightarrow$ *whole relation is one group*

Aggregate Functions

Function	Meaning	NULLs
SUM(A)	Sum of column A	Ignored
AVG(A)	Average of column A	Ignored
COUNT (*)	Number of tuples	Counted
COUNT(A)	Non-NULL values in A	Ignored
MAX(A)	Largest value	Ignored
MIN(A)	Smallest value	Ignored

Key Rules

- Grouping attrs appear unchanged in result schema
- Empty group $\rightarrow$ COUNT=0, all others $\rightarrow$ NULL
- Aggregate results need new attribute names

SQL: `SELECT AVG(gpa) FROM Students`

AGGREGATION IN RELATIONAL ALGEBRA ($\mathcal{G}$)

Syntax

grouping attrs $\mathcal{G}$ agg functions (R)

Omit grouping attrs $\rightarrow$ whole relation is one group

Aggregate Functions

Function	Meaning	NULLs
SUM(A)	Sum of column A	Ignored
AVG(A)	Average of column A	Ignored
COUNT(*)	Number of tuples	Counted
COUNT(A)	Non-NULL values in A	Ignored
MAX(A)	Largest value	Ignored
MIN(A)	Smallest value	Ignored

Key Rules

- Grouping attrs appear unchanged in result schema
- Empty group $\rightarrow$ COUNT=0, all others $\rightarrow$ NULL
- Aggregate results need new attribute names

SQL: SELECT AVG(gpa) FROM Students

$\mathcal{G}$ AVG(gpa) (Students)

$\rightarrow$ One tuple: the overall average GPA

AGGREGATION IN RELATIONAL ALGEBRA ($\mathcal{G}$)

Syntax

grouping attrs $\mathcal{G}$ agg functions (**R**)

Omit grouping attrs $\rightarrow$ whole relation is one group

Aggregate Functions

Function	Meaning	NULLs
SUM(A)	Sum of column A	Ignored
AVG(A)	Average of column A	Ignored
COUNT (*)	Number of tuples	Counted
COUNT(A)	Non-NULL values in A	Ignored
MAX(A)	Largest value	Ignored
MIN(A)	Smallest value	Ignored

Examples

Students(sid, name, major, gpa)

Group by major

major $\mathcal{G}$ AVG(gpa), COUNT(*) (Students)

$\rightarrow$ One row per major: avg GPA and student count

SQL: SELECT major, AVG(gpa), COUNT(*) FROM Students
GROUP BY major

AGGREGATION IN RELATIONAL ALGEBRA ($\mathcal{G}$)

Syntax

grouping attrs $\mathcal{G}$ agg functions (R)

Omit grouping attrs $\rightarrow$ whole relation is one group

Aggregate Functions

Function	Meaning	NULLs
$SUM(A)$	Sum of column A	Ignored
$AVG(A)$	Average of column A	Ignored
$COUNT(*)$	Number of tuples	Counted
$COUNT(A)$	Non-NULL values in A	Ignored
$MAX(A)$	Largest value	Ignored
$MIN(A)$	Smallest value	Ignored

Examples

Students(sid, name, major, gpa)

σ after $\mathcal{G}$, equivalent to HAVING

$\sigma_{\{AVG(gpa) > 3.5\}}(major \ \mathcal{G} \ AVG(gpa) \ (Students))$

$\rightarrow$ Majors where average GPA exceeds 3.5

SQL: SELECT major, AVG(gpa) FROM Students
GROUP BY major HAVING AVG(gpa) > 3.5

Handout exercises

New Content – Views and Indexes!

Example: HR Company

emp_id	name	phone	dept	salary sensitive	ssn sensitive	rating sensitive
1	A. Chen	416-555-0101	Engineering	\$112,000	***-**-4821	4.2
2	M. Patel	416-555-0188	Finance	\$98,500	***-**-7734	3.8
3	J. Osei	416-555-0144	HR	\$87,000	***-**-2290	4.6

Alex works the **front desk** at a company. When someone calls in sick, they look them up to find their **manager's contact info**. The IT department gives them access to the HR database.



Example: HR Company

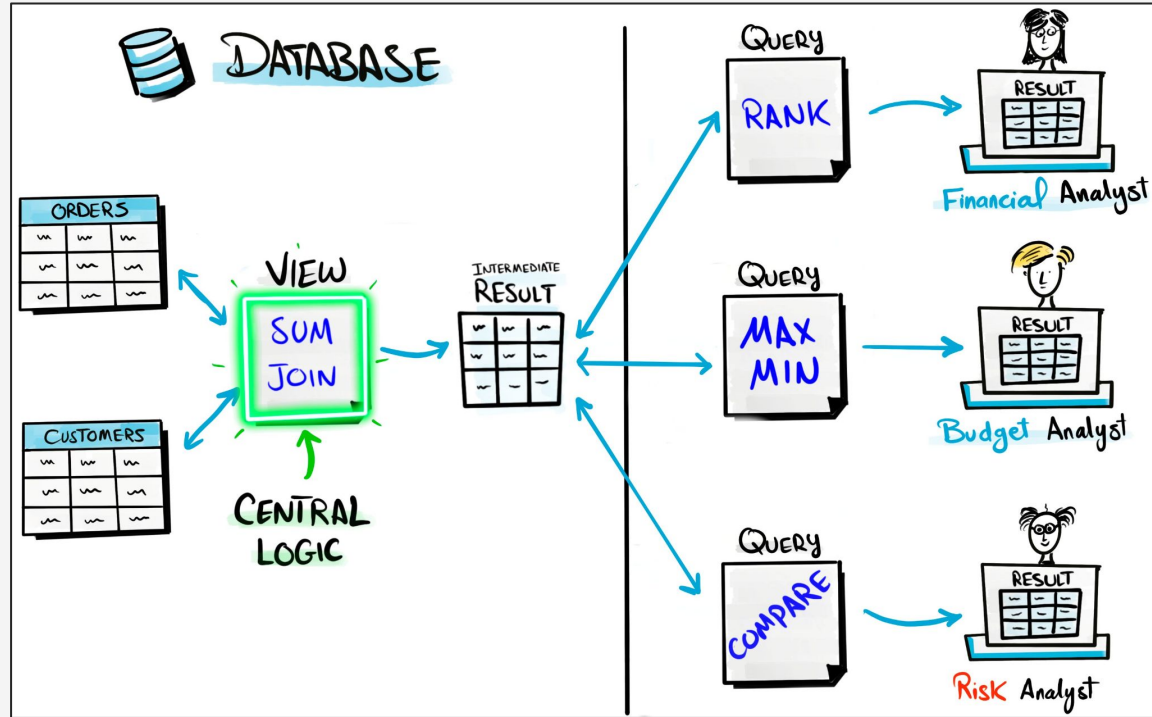
emp_id	name	phone	dept	salary sensitive	ssn sensitive	rating sensitive
1	A. Chen	416-555-0101	Engineering	\$112,000	***-**-4821	4.2
2	M. Patel	416-555-0188	Finance	\$98,500	***-**-7734	3.8
3	J. Osei	416-555-0144	HR	\$87,000	***-**-2290	4.6

Alex works the front desk at a company. When someone calls in sick, they look them up to find their manager's contact info. The IT department gives them access to the HR database.

Turns out, the employees table also has everyone's salary and performance rating. Now Alex knows their boss earns \$180,000 a year.



Views help with this!



Example: HR Company

emp_id	name	phone safe	dept safe
1	A. Chen	416-555-0101	Engineering
2	M. Patel	416-555-0188	Finance
3	J. Osei	416-555-0144	HR

Views: Motivation

Main idea: if it doesn't make sense to make a new table, but not everyone should see everything, then create a view.

- **Security:** A view exposes only safe columns. Grant access to the view instead of a table.
- **Simplicity:** A view can define a 5-table join once, making queries easy.
- **Stability:** Renaming a column doesn't break every query, the view absorbs the name change

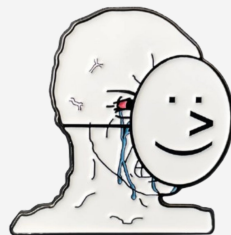
Views: Motivation

Main idea: if it doesn't make sense to make a new table, but not everyone should see everything, then create a view.

- **Security:** A view exposes only safe columns. Grant access to the view instead of a table.
- **Simplicity:** A view can define a 5-table join once, making queries easy.
- **Stability:** Renaming a column doesn't break every query, the view absorbs the name change

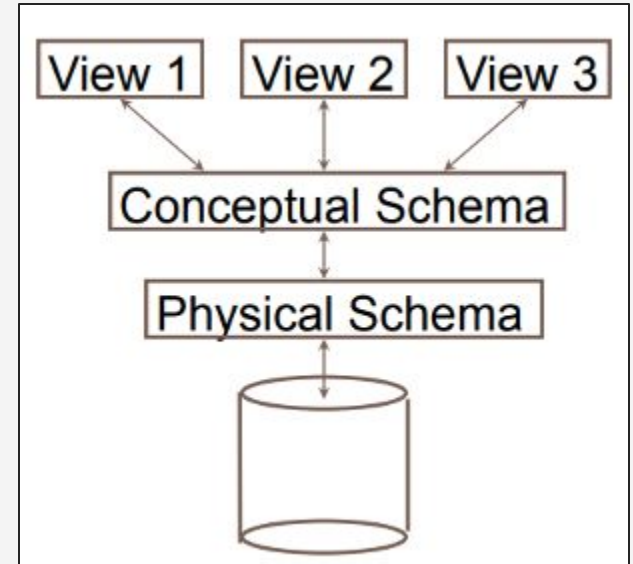
“A view is just a query that pretends to be a table.”

- A wise person, on Reddit.



Levels of Abstraction

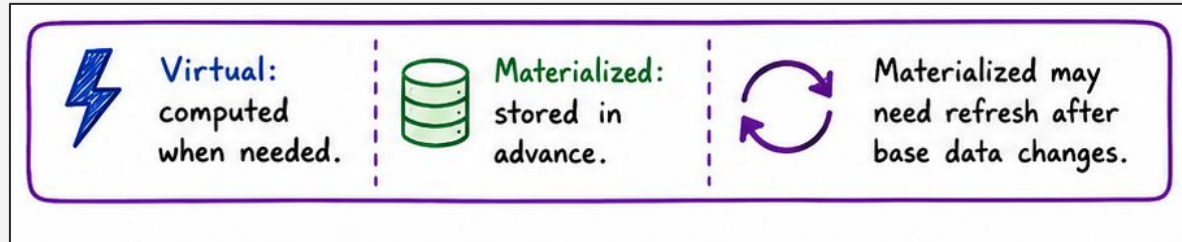
1. Database
2. Physical Schema
 - Describing the files and index(es) used.
3. Conceptual (Logical) Schema
4. Defining the logical structure.
5. **Views**
 - **Describe how users can see the data.**



Views

Two types of views:

- **Virtual:** not stored in the database; it is a query for constructing the relation each time they are accessed.
- **Materialized:** actually constructed and stored; updated periodically depending on the query definition.



VIEWS: Virtual vs Materialized

Virtual View

Base Table: Orders

order_id	customer	amount
1	Alice	50
2	Bob	150
3	Carol	200
4	Dave	80
5	Eve	120

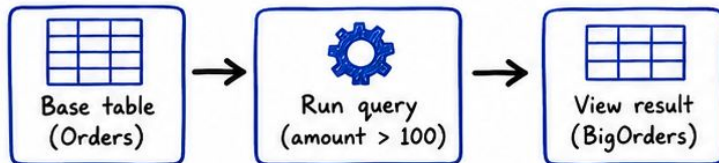
```
CREATE VIEW BigOrders AS  
SELECT *  
FROM Orders  
WHERE amount > 100;
```

View Result: BigOrders

order_id	customer	amount
2	Bob	150
3	Carol	200
5	Eve	120

Computed
when
used!

★ Not stored separately.
Built from the base table when queried.



Materialized View

Base Table: Orders

order_id	customer	amount
1	Alice	50
2	Bob	150
3	Carol	200
4	Dave	80
5	Eve	120

```
CREATE MATERIALIZED VIEW BigOrders AS  
SELECT *  
FROM Orders  
WHERE amount > 100;
```

Materialized View: BigOrders

order_id	customer	amount
2	Bob	150
3	Carol	200
5	Eve	120

Stored
copy

REFRESH

★ After Orders changes,
refresh to update the snapshot.



Declaring Views

Declare by:

```
CREATE [MATERIALIZED] VIEW <name> AS <query>;
```

the DEFAULT view is VIRTUAL

- A view name
- A possible list of attribute names
 - e.g., when arithmetic operations are specified or when we want the names to be different from the attributes in the base relations.
- A query to specify the view contents

Example: View Definitions

CSC343Students(sid, name, grade) is a view “containing” the students that took CSC343.

```
CREATE VIEW CSC343Students AS  
SELECT S.sid, S.name, T.grade  
FROM Student S, Took T  
WHERE S.sid = T.sid  
AND T.cnum = 'CSC343';
```

Example: Accessing a View

Query a view as if it were a base table.

Example Query:

```
SELECT name  
FROM CSC343Students  
WHERE grade >= 80;
```

Updates on Views

Generally, it is impossible to modify a virtual view; because it does not exist.

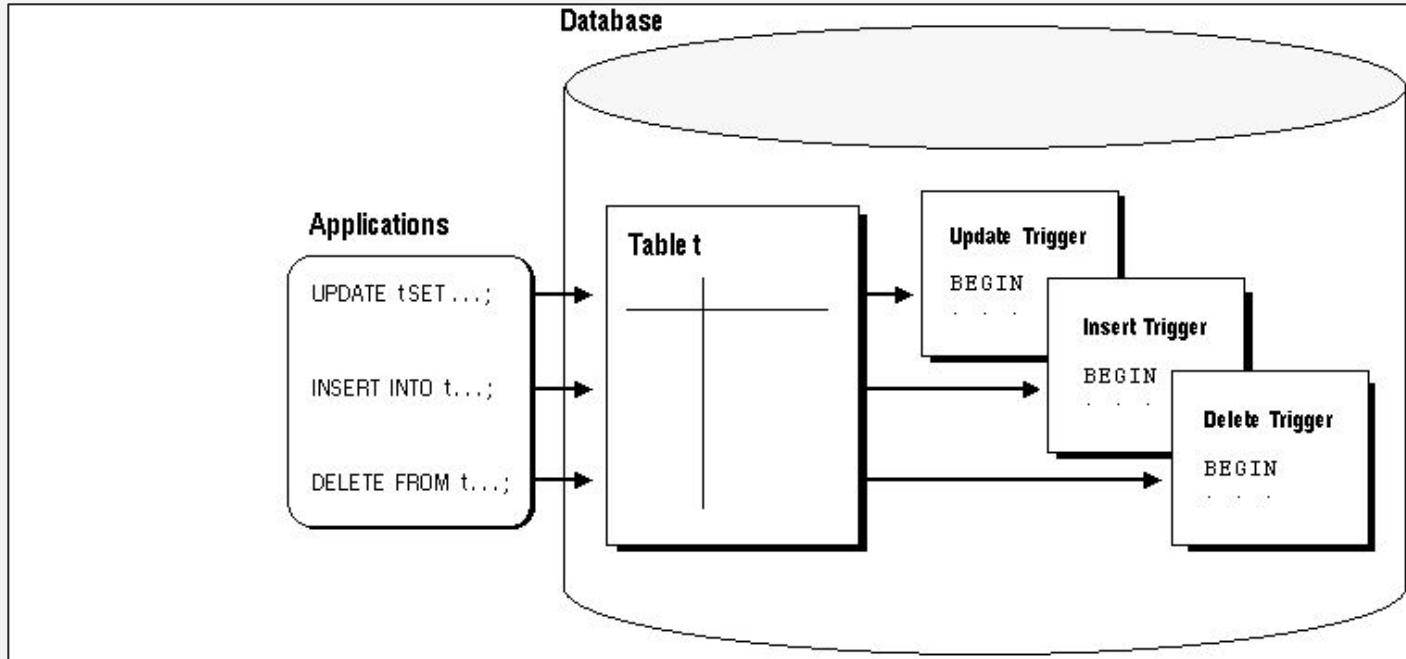
When you query a view, the database can always translate that into a SELECT on the base tables, that's straightforward.

But when you try to write through a view (INSERT, UPDATE, DELETE), the database has to figure out what that would mean on the underlying tables. And often, it just can't.

The rule of thumb is: a view is only updatable if the database can unambiguously trace every column back to exactly one row in exactly one base table. Most real views don't meet that bar, which is why most systems block updates by default.

Triggers

A trigger is a stored procedure that the database runs automatically in response to an event, you don't call it, the database does.



TRIGGER

1. INSERT into Orders

Orders

order_id	customer
101	Alice
102	Bob
103	Cara
104	Dan



3. add row to Audit

Audit

order_id	action
101	INSERT
102	INSERT
103	INSERT
104	INSERT

Trigger code

```
CREATE TRIGGER log_order  
AFTER INSERT ON Orders  
FOR EACH ROW  
INSERT INTO Audit  
VALUES (NEW.order_id, 'INSERT');
```



When an event happens,
SQL automatically does
another action.

Example:

```
CREATE TRIGGER LogEnrollment
AFTER INSERT ON Took
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog VALUES (NEW.sid, 'enrolled',
CURRENT_TIMESTAMP);
END;
```

This is
experimental
with me trying
nano banana...
OML

TRIGGER: Proactive Event Logging

1. EVENT: INSERT on Took

Took		
sid	cnum	grade
101	CSC343	A
102	CSC343	B
103	CSC343	C
NEW	123	CSC343 A



2. ACTION: Log to AuditLog

AuditLog		
sid	action	timestamp
101	enrolled	...
102	enrolled	...
103	enrolled	...
123	enrolled	10:00

NEW.sid, 'enrolled',
CURRENT_TIMESTAMP

Trigger code

```
CREATE TRIGGER LogEnrollment
AFTER INSERT ON Took
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog VALUES(NEW.sid, 'enrolled', CURRENT_TIMESTAMP);
END;
```

Trigger Syntax

```
CREATE TRIGGER TriggerName  
AFTER INSERT ON TableName  
REFERENCING NEW ROW AS n  
FOR EACH ROW BEGIN  
    -- trigger body
```

```
END;
```

the event: *when* does this fire and *on what action*

Triggers can fire before, after, and instead of.

Trigger Syntax

```
CREATE TRIGGER TriggerName  
AFTER INSERT ON TableName  
REFERENCING NEW ROW AS n  
FOR EACH ROW BEGIN  
    -- trigger body  
  
END;
```

gives the new row a name you can use in the body

Trigger Syntax

```
CREATE TRIGGER TriggerName  
AFTER INSERT ON TableName  
REFERENCING NEW ROW AS n  
FOR EACH ROW BEGIN  
    -- trigger body
```

```
END;
```

runs once per affected row (vs. once per statement)

Update Triggers

```
CREATE TRIGGER CheckGradeUpdate
AFTER UPDATE ON Took
REFERENCING OLD ROW AS o NEW ROW AS n
FOR EACH ROW BEGIN
    INSERT INTO GradeLog(sid, cnum, old_grade, new_grade,
changed_at) VALUES(o.sid, o.cnum, o.grade, n.grade,
CURRENT_TIMESTAMP);

END;
```

Update triggers can look at both the old and new row values

```
UPDATE Took SET grade = 88 WHERE sid = 1234 AND cnum =
'CSC343';
```

Statement vs. Row Level Triggers

```
CREATE TRIGGER RowLog
AFTER INSERT ON Took
REFERENCING NEW ROW AS n
FOR EACH ROW BEGIN
    INSERT INTO Log
    VALUES (n.sid, n.grade);
END;
```

```
CREATE TRIGGER StmtLog
AFTER INSERT ON Took
FOR EACH STATEMENT BEGIN
    INSERT INTO Log
    VALUES ('batch insert
done');
END;
```

Triggers can run for each row, and also each statement.

```
INSERT INTO Took(sid, cnum, grade) VALUES (1001, 'CSC343',
85), (1002, 'CSC343', 91), (1003, 'CSC343', 78);
```

Visual Example

```
INSERT INTO Orders(order_id, customer)
VALUES (104, 'Dan'),
       (105, 'Eve'),
       (106, 'Farah');
```

One SQL statement inserts 3 rows.

FOR EACH ROW

```
CREATE TRIGGER log_each_order
AFTER INSERT ON Orders
FOR EACH ROW
INSERT INTO Audit(order_id, action)
VALUES (NEW.order_id, 'INSERT');
```

Orders (after insert)

order_id	customer
104	Dan
105	Eve
106	Farah



Audit

order_id	action
104	INSERT
105	INSERT
106	INSERT

★ **fires 3 times**

- once per affected row

FOR EACH STATEMENT

```
CREATE TRIGGER log_insert_statement
AFTER INSERT ON Orders
FOR EACH STATEMENT
INSERT INTO AuditSummary(action)
VALUES ('INSERT statement ran');
```

Orders (after insert)

order_id	customer
104	Dan
105	Eve
106	Farah



Audit Summary

action
INSERT statement ran

★ **fires 1 time**

- once for the whole SQL statement

Review Time (Exercises)

Interpreting a View Insertion

```
CREATE TRIGGER ViewTrig
INSTEAD OF INSERT ON StudentCourses
REFERENCING NEW ROW AS n
FOR EACH ROW
BEGIN
    INSERT INTO Took VALUES (n.sid, n.cnum);
    INSERT INTO Offers VALUES (n.cnum, n.dept);
    INSERT INTO EnrolledIn VALUES (n.sid, n.dept);
END;
```

Recall: the columns need to be unambiguous for a View insert.